2 академических часа · студенты-программисты

Docker Images. Образы и реестры

Лекция №3 · Docker и контейнеризация приложений

Цель лекции

- Понять устройство образов Docker, научиться выбирать, получать, исследовать, маркировать и публиковать образы.
- Образ — шаблон, контейнер — выполняемый экземпляр.
- Образы поставляются через реестры.
- Теги упрощают управление версиями, но могут изменяться.

Что мы уже умеем

- После двух первых занятий студент умеет:
- Запускать и останавливать контейнеры.
- Работать с docker run, ps, exec, logs.
- Проверять службу Docker в Debian 13.

Учебные вопросы

- 1. Понятие и назначение Docker Image.
- 2. Структура образа, слои и идентификаторы.
- 3. Реестры и выбор безопасных образов.
- 4. Локальное управление образами.
- 5. Теги, версии и публикация в Docker Hub.

Понятие и назначение Docker Image

Образ как воспроизводимый шаблон контейнера

Что такое Docker Image?

- Docker Image — неизменяемый шаблон, включающий файловую систему и метаданные, используемые для создания контейнера.
- Образ может содержать программу, библиотеки и файлы конфигурации.
- Сам образ не является запущенным процессом.
- Из одного образа можно создавать много контейнеров.

Образ и контейнер

Docker Image

- Шаблон с файловой системой и метаданными.
- Используется при создании контейнера.
- Может иметь несколько тегов.

Docker Container

- Экземпляр, созданный из образа.
- Имеет собственное состояние и записываемый слой.
- Может быть запущен, остановлен или удалён.

Один образ — несколько контейнеров

Как один Docker-образ используется для создания нескольких независимых контейнеров

Docker Image

nginx:stable

В образе содержится:

- Файлы приложения
- Библиотеки
- Конфигурация
- Метаданные

Образ — это шаблон
Контейнер — это запущенный экземпляр образа



Контейнер web1

- Собственный writable layer
- Свой процесс
- Порт 8081 → 80



Контейнер web2

- Собственный writable layer
- Свой процесс
- Порт 8082 → 80



Контейнер web3

- Собственный writable layer
- Свой процесс
- Порт 8083 → 80

Что важно понять

- Один образ можно использовать много раз.
- Каждый контейнер изолирован от других.
- Остановка одного контейнера не влияет на остальные.
- Удаление контейнера не удаляет образ автоматически.

Кратко: один образ — несколько контейнеров



Для чего программисту нужен образ?

- Образ делает поставку среды выполнения повторяемой.
- Одинаковая версия приложения и зависимостей.
- Передача сборки между разработчиком и сервером.
- Повторное использование базовых образов.
- Масштабирование количества контейнеров.

Пример: образ Nginx

- nginx:stable — ссылка на готовый образ Nginx с тегом stable.
- Образ можно скачать, не запуская контейнер.
- При docker run из него создаётся контейнер.
- Тег stable может со временем указывать на обновлённый образ.

Скачивание образа без запуска

```bash
sudo docker pull nginx:stable
```

Что важно

- pull получает образ из реестра.
- nginx — репозиторий образа, stable — тег.
- При успешной загрузке видны скачанные слои и итоговая ссылка на образ.

Структура образа, слои и идентификаторы

Почему образы эффективно хранятся и передаются

Слои образа

- Файловая система образа представлена слоями, которые могут совместно использоваться несколькими образами.
- Слои позволяют повторно использовать уже имеющееся содержимое.
- При загрузке Docker получает недостающие слои.
- Создание контейнера не изменяет исходные слои образа.

Слои Docker-образа

Dockerfile (пример)

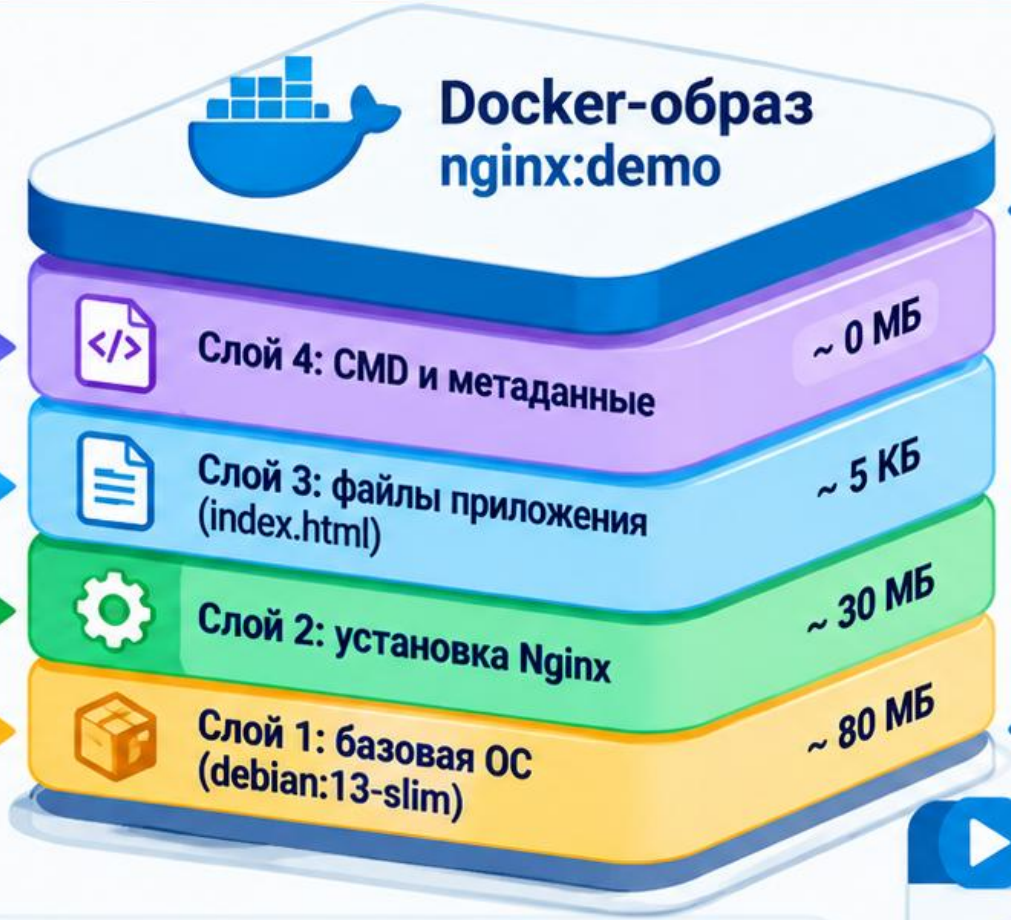
```
FROM debian:13-slim

RUN apt update && \
    apt install -y nginx

COPY index.html \
     /var/www/html/

EXPOSE 80

CMD ["nginx", "-g",
     "daemon off;"]
```



Что такое слои?

- Образ состоит из нескольких слоёв (layers).
- Каждый слой — это изменения в файловой системе.
- Слои неизменяемы (read-only) и могут использоватся повторно в других образах.
- При загрузке Docker скачивает только недостающие слои.

Повторное использование слоёв



Создание контейнера



Образ
(только чтение)

Записываемый слой контейнера (изменения)

Контейнер
(собственное состояние)

Записываемый слой контейнера

- При создании контейнера Docker добавляет поверх слоёв образа записываемый слой.
- Изменения внутри контейнера не переписывают исходный образ.
- Удаление контейнера обычно удаляет его записываемый слой.
- Для постоянных данных позднее используем volumes.

История формирования образа

```bash
sudo docker history nginx:stable
```

Что важно

- history показывает историю инструкций и размер соответствующих записей.
- Не каждая строка истории обязательно означает отдельный файловый слой.
- Сравните размеры и обратите внимание на <missing> в некоторых записях.

Исследование метаданных

```bash
sudo docker image inspect nginx:stable
```

Что важно

- inspect возвращает JSON с метаданными образа.
- Найдите Id, RepoTags, RepoDigests, Architecture, Os, Config.
- Команда не запускает контейнер.

Image ID, digest и тег

- Image ID идентифицирует локальный объект образа; digest идентифицирует конкретное содержимое опубликованного артефакта в реестре; тег — удобное изменяемое имя.
- Теги могут быть переназначены на другой образ.
- Digest sha256 позволяет закреплять конкретную опубликованную версию.
- Не следует считать latest синонимом самой новой версии приложения.

Посмотреть локальные ссылки и идентификаторы

```bash
sudo docker image ls --digests
```

Что важно

- В таблице будут REPOSITORY, TAG, DIGEST, IMAGE ID и SIZE.
- У некоторых образов локально digest может отсутствовать.
- Сравните IMAGE ID и имя с тегом.

Реестры и выбор образов

Docker Hub и другие контейнерные реестры

Что такое Registry?

- Container registry — сервис для хранения и передачи образов.
- Docker Hub — распространённый публичный реестр.
- Реестры также могут быть частными или корпоративными.
- Образы получают через pull, публикуют через push.

Локальный Docker и реестр образов

Образы можно хранить локально на компьютере и загружать из удалённого реестра или публиковать свои образы

Локальный компьютер

Docker CLI

Команды:
`docker pull`
`docker push`
`docker images` и др.

Docker Engine (Daemon)

Управляет образами и контейнерами

Локальное хранилище образов

- nginx:latest
- node:20
- ubuntu:22.04
- myapp:v1

Загрузка образа
`docker pull`

Публикация образа
`docker push`

Интернет

Удалённый реестр образов

Например, Docker Hub

Docker Hub

hub.docker.com

Публичный реестр образов

Примеры официальных образов

nginx	`docker pull nginx`
node	`docker pull node`
ubuntu	`docker pull ubuntu`
redis	`docker pull redis`

Загрузка образа из реестра

```
# Скачать образ nginx
docker pull nginx

# Проверить локальные образы
docker images
```

Публикация своего образа

```
# Войти в Docker Hub
docker login
# Отметить образ своим тегом
docker tag myapp:1.0 username/myapp:1.0
# Отправить образ в реестр
docker push username/myapp:1.0
```

Репозиторий, namespace и тег

- Пример ссылки: docker.io/library/nginx:stable.
- docker.io — адрес реестра.
- library — namespace официальных образов Docker Hub.
- nginx — репозиторий, stable — тег.
- При отсутствии реестра Docker по умолчанию использует Docker Hub.

Примеры ссылок на образы

Ссылка	Реестр	Назначение
nginx:stable	Docker Hub	Nginx
postgres:17	Docker Hub	PostgreSQL
node:22	Docker Hub	Node.js
ghcr.io/owner/app:v1	GitHub Container Registry	Пример пользовательского репозитория

Выбор образа для проекта

- Перед использованием изучите описание и происхождение образа.
- Предпочитайте официальные или проверенные издателем образы.
- Проверяйте поддерживаемые версии и архитектуру.
- Читайте инструкции по переменным окружения и томам.
- Обновляйте образы и отслеживайте уязвимости.

Тег latest: частое заблуждение

- latest — просто тег, который используется по умолчанию, если тег не указан.
- Его значение задаёт владелец репозитория.
- Он не гарантирует последнюю стабильную версию ПО.
- В учебном проекте лучше указывать осмысленный тег версии.

Локальное управление образами

Команды получения, анализа и удаления

Показать список образов

```bash
sudo docker image ls
```

Что важно

- Отображает локальные образы с именами, тегами и размерами.
- Альтернативная команда: sudo docker images.
- После pull образ появится в списке.

Скачать определённую версию

```bash
sudo docker pull alpine:3.21
```

Что важно

- Получает образ с конкретным тегом.
- Если образ для платформы доступен, Docker сохранит его локально.
- Запуск контейнера для скачивания не нужен.

Запустить контейнер из скачанного образа

```bash
sudo docker run --rm alpine:3.21 echo "Hello from Alpine"
```

Что важно

- run создаёт и запускает контейнер.
- --rm удаляет контейнер после завершения команды.
- echo выводит строку, после чего процесс завершается.

Проверить образ по идентификатору

```bash
sudo docker image inspect alpine:3.21 --format "{{.Id}}"
```

Что важно

- --format выводит одно заданное поле из JSON.
- Появится локальный идентификатор образа sha256:...
- Вывод удобно использовать в диагностике.

Переименовать локальную ссылку через tag

BASH

```
sudo docker tag alpine:3.21 classroom/alpine:lab1
```

Что важно

- tag добавляет ссылку на существующий образ.
- Команда не пересобирает образ и не создаёт его копию.
- Проверьте, что оба имени ведут к одному IMAGE ID.

Удаление только созданного тега

```bash
sudo docker rmi classroom/alpine:lab1
```

Что важно

- rmi удаляет указанную локальную ссылку.
- Если образ имеет другие теги, он продолжает быть доступным.
- После команды alpine:3.21 должен остаться.

Почему образ может не удаляться?

- Docker защищает используемые образы от удаления, способного нарушить зависимости.
- Образ может быть связан с существующим контейнером.
- У одного образа может быть несколько тегов.
- Не используйте принудительную очистку чужих учебных данных.

Аккуратная диагностика перед удалением

```bash
sudo docker ps -a
sudo docker image ls
```

Что важно

- Сначала выясните, какие контейнеры существуют.
- После этого проверьте локальные образы и теги.
- Удаляйте только объекты, созданные в текущей работе.

Теги, версии и публикация в Docker Hub

От локального имени к доступному другим образу

Зачем нужны теги?

- Тег помогает обозначить выпуск или вариант образа.
- Примеры: v1, v1.1, test, stable.
- Один образ может иметь несколько ссылок и тегов.
- Переназначение тега возможно, если реестр не запрещает его менять.

Тег, Image ID и Digest

Один и тот же образ может иметь разные теги. Тег — удобное имя, Image ID — локальный идентификатор, digest — уникальная контрольная сумма содержимого образа.



Тег (Tag)

Человекочитаемое имя образа.

- Состоит из имени репозитория и версии.
- Удобен для использования и указания версии.
- Один и тот же образ может иметь несколько тегов.

Примеры тегов:

`nginx:latest` `nginx:1.25` `node:20` `ubuntu:22.04`

Image ID

Локальный идентификатор образа.

- Короткий хэш (первые 12 символов) полного хэша конфигурации образа.
- Один и тот же для всех тегов, указывающих на один и тот же образ.
- Используется Docker для внутренней идентификации и хранения образов.

Пример (полный и сокращённый):

```
3f2a5c7d9e1b                                  # сокращённый (12 символов)
3f2a5c7d9e1b8c4d6e7f9a1b2c3d4e5f6g7h8i3j0k112n3n4o5p6   # полный (64 символа)
```

Digest

Уникальная контрольная сумма содержимого образа.

- Вычисляется на основе содержимого (слоёв) образа.
- Не зависит от тега.
- Позволяет точно определить, какой именно образ используется.
- Используется для безопасного и воспроизводимого развёртывания.

Пример digest:

```
sha256:8d3f4c2e1a9b7c6d5e4f3a2b1c0d9e8f7a6b5c4d3e2f1a0b9c8d7e6f5a4b3c2d1e0f
```

Сравнение

Характеристика	Тег (Tag)	Image ID	Digest
Назначение	Человекочитаемое имя и версия	Локальный идентификатор	Уникальная контрольная сумма
Формат	repo:tag (например, nginx:1.25)	Короткий хэш (12 символов)	sha256:... (64 символа)
Может изменяться	Да (можно создать новый тег)	Нет (меняется при изменении образа)	Нет (зависит от содержимого)
Один образ — несколько значений	Может иметь несколько тегов	Один и тот же для всех тегов	Один и тот же для всех тегов

Пример подготовки имени для публикации

```bash
sudo docker tag nginx:stable student123/docker-nginx-lab:v1
```

Что важно

- student123 — условный Docker Hub username: замените на собственный.
- docker-nginx-lab — имя репозитория.
- v1 — тег учебного выпуска.

Подготовка к Docker Hub

- Для публикации необходимо иметь учётную запись и доступ на запись в выбранный репозиторий.
- Создайте собственный репозиторий Docker Hub или используйте доступный вам.
- Не записывайте пароли и токены в отчёт.
- Для общего компьютерного класса выполняйте logout после публикации.

Аутентификация в Docker Hub

```bash
sudo docker login
```

Что важно

- Docker CLI запрашивает или организует аутентификацию по поддерживаемому сценарию.
- При использовании sudo учётные данные сохраняются в контексте root.
- Никогда не показывайте токен преподавателю на скриншоте.

Публикация образа

```bash
sudo docker push student123/docker-nginx-lab:v1
```

Что важно

- Замените student123 на собственный Docker Hub username.
- В реестр отправляются необходимые слои и метаданные.
- Проверьте появление тега v1 в Docker Hub.

Публикация образа в Docker Hub

Создаём образ, присваиваем тег, загружаем в Docker Hub и проверяем результат

1 Создаём локальный образ

Например, собираем образ из Dockerfile или используем готовый образ.

Dockerfile

```
FROM node:20
WORKDIR /app
COPY . .
RUN npm install
CMD ["node", "app.js"]
```

myapp:1.0
Локальный образ
на вашем компьютере

2 Входим в Docker Hub

Выполняем авторизацию в своей учётной записи Docker Hub.

```
> docker login
Username: yourname
Password: ********\
Login Succeeded
```

ℹ Нужна учётная запись на hub.docker.com (бесплатная).

3 Присваиваем тег для публикации

Тег должен содержать имя вашего репозитория в Docker Hub.
Формат: `<username>/<image>:<tag>`

```
> docker tag myapp:1.0 \\
  yourname/myapp:1.0
```

myapp:1.0
Локальный тег

yourname/myapp:1.0
Тег для Docker Hub
(тот же образ)

4 Загружаем образ в Docker Hub

Отправляем образ в удалённый реестр.

```
> docker push
  yourname/myapp:1.0

The push refers to repository
[...]
Pushed
Pushed
Pushed
...
latest: digest: sha256:8d3f...
```

5 Проверяем результат

Образ доступен в вашем репозитории на Docker Hub.

Docker Hub
hub.docker.com

yourname/myapp

1.0	2 минуты назад
latest	2 минуты назад
0.9	3 дня назад
...	

Полный набор команд

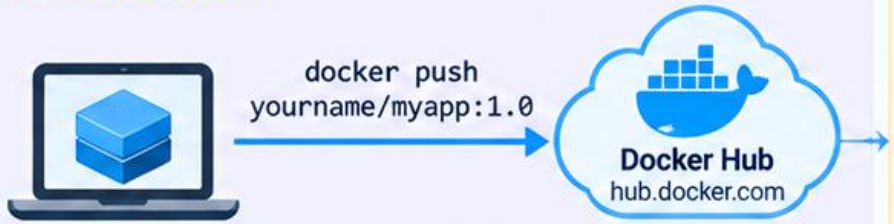
```
# 1. Сборка образа
docker build -t myapp:1.0 .

# 2. Вход в Docker Hub
docker login

# 3. Присвоение тега
docker tag myapp:1.0 yourname/myapp:1.0

# 4. Публикация
docker push yourname/myapp:1.0
```

Схема процесса



Локальный образ
myapp:1.0

Репозиторий
yourname/myapp

Важные моменты

1. Имя репозитория в формате `<username>/<image>`.
2. Тег можно выбрать любой (например, `1.0`, `latest`).
3. Для публикации требуется авторизация в Docker Hub.
4. Один и тот же образ можно загружать с разными тегами.
5. Публичный репозиторий доступен всем, приватный — только вам и приглашённым пользователям.

Получение опубликованного образа

BASH

```
sudo docker pull student123/docker-nginx-lab:v1
```

Что важно

- Команда скачивает доступный вам образ из Docker Hub.
- Для приватного репозитория может потребоваться авторизация.
- Проверка публикации завершается появлением образа в списке.

Тег и Digest

Тег

- Удобное читаемое имя: v1, stable.
- Может переназначаться.
- Полезен для выпусков и каналов обновлений.

Digest

- Криптографический идентификатор содержимого.
- Используется для точного закрепления опубликованного артефакта.
- Меняется при изменении идентифицируемого содержимого.

Частые ошибки студентов

- Типовые проблемы при работе с образами:
- pull: неверно указаны имя или тег, нет сети.
- push: нет авторизации или прав на namespace.
- rmi: образ используется контейнером.
- latest: ошибочное ожидание фиксированной версии.

Алгоритм диагностики ошибок pull / push

1. **Шаг 1** — Проверить доступность сети и правильность имени реестра.
2. **Шаг 2** — Проверить репозиторий и указанный тег.
3. **Шаг 3** — Для push проверить свою авторизацию и права на репозиторий.
4. **Шаг 4** — Изучить точное сообщение об ошибке в терминале.
5. **Шаг 5** — Повторить операцию и подтвердить результат через image ls или Docker Hub.

Связь с итоговым проектом

- Итоговое приложение будет использовать готовые образы Nginx и PostgreSQL, а собственный образ Express API мы соберём позднее.
- Лекция №4: Dockerfile и сборка своего образа.
- Лекция №7: управление сервисами через Docker Compose.
- Лекция №12: передача образа на Linux-сервер.

Итоги лекции №3

1. Образ — шаблон, контейнер — экземпляр с отдельным состоянием.
2. Слои образов могут повторно использоваться.
3. pull скачивает, tag добавляет ссылку, push публикует, rmi удаляет локальную ссылку.
4. Тег не обязательно неизменен; для точной фиксации применяют digest.
5. Публикация в Docker Hub требует подходящих прав.

Контрольные вопросы

1. Чем образ отличается от контейнера?
2. Для чего нужны слои образа?
3. Чем отличаются repository, tag и digest?
4. Что покажет docker history?
5. Почему docker tag не копирует образ?
6. Для чего нужны docker login и docker push?
7. Почему latest не гарантирует фиксированную версию?
8. Как безопасно удалить учебный тег образа?